

Guía para organizar el Dataset_RiwiSupply_SUCIO.xlsx

Revisé tu archivo: tiene **90 filas × 27 columnas** en la hoja `Dataset_RiwiSupply`, y ya incluye una hoja `Leyenda_Inconsistencias` que cataloga 15 tipos de problemas. Esta guía te lleva paso a paso desde ese archivo "sucio" hasta un conjunto de tablas limpias y normalizadas (las mismas `riwi_*` del simulacro).

Hay dos niveles de trabajo, y conviene hacerlos en este orden:

Nivel 1 — Limpieza de datos (arreglar valores dentro de la tabla plana) **Nivel 2 — Normalización** (separar la tabla plana en varias tablas relacionadas)

Nivel 1 • Limpieza de datos

1. Eliminar filas basura

- **Filas totalmente vacías:** detecté las filas 87-90 (`id_registro` 87-90) con casi todo en NULL excepto `pais_proveedor`. Bórralas.
- **Filas duplicadas exactas:** las filas con `id_registro = 22` y `id_registro = 28` aparecen repetidas dos veces cada una (filas 81/84 y 83/85). Elimina la copia repetida, no el registro completo.
- Regla general: si una fila tiene más del 80% de campos clave vacíos y no aporta un `id_registro` nuevo/consistente, es candidata a eliminar.

2. Estandarizar texto (nombres, ciudades, categorías, tipo de movimiento, unidad de medida, país)

El mismo valor real aparece escrito de muchas formas. Ejemplos reales de tu archivo:

- Proveedor: `Tecno Partes Ltda.` / `TECNOPARTES LTDA` / `Tecnopartes Ltda` → mismo proveedor (mismo NIT `700345678-5`).
- Ciudad: `Bogotá` / `Bogota` / `BOGOTA D.C.` / `BOGOTÁ` → mismo valor.
- Unidad de medida: `mts` / `mt` / `ML` / `Metro` → mismo valor.
- Tipo de movimiento: `Entrada` / `entrada` / `INGRESO` / `ingreso` → mismo valor (solo dos categorías reales: ENTRADA/SALIDA).
- País: `Colombia` / `CO` / `Col` / `colombia` → mismo valor.

Cómo hacerlo:

1. Pasa todo el texto a un formato canónico: `TRIM` + mayúsculas/minúsculas consistentes (`PROPER` o `UPPER`).

2. Crea una **tabla de equivalencias** (diccionario) por cada columna problemática, por ejemplo:

Valor encontrado	Valor estándar
Bogota, BOGOTA D.C., BOGOTÁ	Bogotá
ingreso, Ingreso, INGRESO	ENTRADA
mt, mts, ML, Metro	m

3. Usa `BUSCARV`/`VLOOKUP` (Excel) o `.replace()`/`.map()` (Python/pandas) contra esa tabla para reemplazar los valores.
4. La forma más confiable de identificar que dos filas son "el mismo proveedor" no es el nombre (que varía), sino una **llave estable**: el `nit_proveedor` o el `id_proveedor`. Usa esa llave para agrupar y luego elige un único nombre "oficial" por grupo.

3. Formatos de fecha inconsistentes (`fecha_compra`)

Encontré al menos 4 formatos distintos mezclados: `2023-07-12`, `25-08-2023`, `27 Apr 2024`, `04/11/2024`.

- Detecta el patrón de cada valor (ISO `AAAA-MM-DD`, `DD-MM-AAAA`, `DD/MM/AAAA`, texto `DD Mon AAAA`).
- Convierte todo a un único formato estándar, recomendado ISO `AAAA-MM-DD`, usando una función de parseo flexible (`pd.to_datetime(..., dayfirst=True)` en Python, o `TEXTO`/`FECHA` combinadas en Excel).
- Ojo con la ambigüedad `04/11/2024`: puede ser 4-nov o 11-abr. Revisa el contexto (rango de fechas del dataset) para decidir la convención (aquí es `DD/MM/AAAA`, formato colombiano).

4. Teléfonos con formatos distintos

`3101234567` vs `310-123-4567` vs `310 123 4567` vs `601-456-7890`. Estandariza quitando espacios/guiones y deja solo dígitos (o un formato fijo `XXX-XXX-XXXX`).

5. Nulos en campos obligatorios

Columnas con NULL: `precio_unitario` (10), `email_proveedor` (19), `nit_proveedor` (9), `fecha_compra` (12), etc.

- Si el dato se puede **recuperar** de otra fila del mismo proveedor/producto (p. ej. el mismo `id_proveedor` sí tiene NIT en otra fila), complétalo por referencia cruzada.

- Si no se puede recuperar, decide una regla explícita: dejarlo NULL solo si la columna lo permite, o marcarlo como ("Sin dato") — nunca inventar un valor.
- Campos que van a ser NOT NULL en la base de datos final (como `precio_unitario`) deben quedar resueltos antes de migrar.

6. Valores calculados incorrectos (`valor_total_compra`)

La leyenda indica que `valor_total_compra` no siempre coincide con `precio_unitario × cantidad_comprada`. Recalcúlalo siempre con fórmula en vez de confiar en el valor cargado:

```
valor_total_compra = precio_unitario * cantidad_comprada
```

Esto también sirve para detectar y rellenar los `precio_unitario` faltantes cuando tengas el total y la cantidad.

Nivel 2 • Normalización (pasar de 1 tabla a varias tablas relacionadas)

La hoja de leyenda ya lo dice directamente: *"Proveedores, productos, bodegas y movimientos están mezclados en una sola tabla"* y hay *"dependencias transitivas"* (p. ej. `ciudad_bodega` depende de la bodega, no del registro de compra). Esto es exactamente el problema de 1FN/2FN/3FN del simulacro. La solución es separar la tabla ancha en entidades independientes:

Tabla nueva	Columnas que salen de la tabla sucia	Clave
riwi_cities	ciudad_proveedor, ciudad_bodega, pais_proveedor (ya estandarizados)	id autogenerado
riwi_suppliers	id_proveedor, nombre_proveedor, nit_proveedor, telefono_proveedor, email_proveedor, city_id (FK)	id_proveedor
riwi_categories	categoria_producto, subcategoria_producto	id autogenerado
riwi_products	id_producto, nombre_producto, descripcion_producto, unidad_medida, precio_unitario, category_id (FK)	id_producto
riwi_warehouses	id_bodega, nombre_bodega, ciudad_bodega (→ FK ciudad), direccion_bodega, responsable_bodega	id_bodega
riwi_purchases	id_registro, fecha_compra, supplier_id (FK), cantidad_comprada, valor_total_compra	id_registro
riwi_inventory_movements	tipo_movimiento, cantidad_movimiento, stock_actual, product_id (FK), warehouse_id (FK), observaciones	id autogenerado

Pasos prácticos:

1. Primero limpia (Nivel 1) — normalizar sobre datos sucios solo mueve el problema a varias tablas en vez de una.
2. Extrae cada entidad con valores **únicos**: por ejemplo para riwi_suppliers, toma id_proveedor + columnas de proveedor, elimina duplicados ((DISTINCT) / drop_duplicates()), y verifica que un mismo id_proveedor no tenga dos nombres distintos después de la limpieza.

3. Asigna un `(id)` propio a las tablas que no lo tenían (`(riwi_cities)`, `(riwi_categories)`, `(riwi_inventory_movements)`).
 4. Reemplaza en la tabla de hechos (`(riwi_purchases)`) los datos descriptivos por las llaves foráneas correspondientes (`(supplier_id)`, `(product_id)`, etc.), eliminando las columnas repetidas.
 5. Valida integridad referencial: todo `(supplier_id)` en `(riwi_purchases)` debe existir en `(riwi_suppliers)`, todo `(product_id)` en `(riwi_inventory_movements)` debe existir en `(riwi_products)`, etc.
-

Herramientas sugeridas según tu flujo de trabajo

- **Solo Excel:** usa Power Query (Datos → Obtener y transformar) para limpieza de texto, `(BUSCARV)` para reemplazos, y Tablas Dinámicas o "Quitar duplicados" para separar entidades.
- **Python (pandas):** es lo más rápido para este volumen y deja el proceso reproducible (`(str.strip())`, `(str.upper())`, `(.replace())`, `(pd.to_datetime())`, `(drop_duplicates())`, `(groupby())`).
- **SQL:** una vez migrados los datos a una base relacional, aplica las validaciones de integridad (`(FOREIGN KEY)`, `(CHECK)`) para que la base rechace inconsistencias futuras — esta es la fase de las tablas `(riwi_*)` que ya usaste en el simulacro.

Checklist final antes de dar por limpio el dataset

- ☐ Sin filas vacías ni duplicadas
- ☐ Todo el texto estandarizado (mismo valor = misma escritura exacta)
- ☐ Fechas en un solo formato
- ☐ Teléfonos en un solo formato
- ☐ Sin nulos en campos obligatorios (o regla explícita documentada)
- ☐ `(valor_total_compra)` recalculado y consistente
- ☐ Datos separados en las 7 tablas `(riwi_*)`, sin columnas repetidas entre ellas
- ☐ Todas las llaves foráneas apuntan a un registro existente

Si quieres, puedo tomar tu archivo y hacer la limpieza real (Nivel 1) y/o generar ya las 7 tablas normalizadas (Nivel 2) como un nuevo Excel — dime cuál de las dos prefieres primero.